Azure Multi-Tier DevOps Lab Guide: Jenkins, Puppet r10k, & Job Chains

Scope: 3-Job Jenkins Pipeline, Apache2 Webserver, Tomcat Appserver, Maven Hello World JAR, GitOps with r10k, GitHub Webhooks, and SSH Security.

1. Azure Prerequisites & Environment Setup

Deploy two Ubuntu 22.04 LTS virtual machines in Azure under the same Virtual Network (VNet):

• **VM 1 (Jenkins Controller):** Standard_B2s, Inbound open ports: 22 (SSH) and 8080 (Jenkins UI).

• **VM 2 (Puppet Server & Target):** Standard_B2s, Inbound open ports: 22, 80 (Apache), 8080 (Tomcat).

• **Installed Packages on VM 1:** Java OpenJDK 17, Jenkins, Git, Maven.

• **Installed Packages on VM 2:** Puppet Server 8, r10k gem, Git, Apache2, Tomcat.

2. SSH Connectivity Configuration between Jenkins and Puppet Agents

To allow Jenkins (VM 1) to securely execute commands on Puppet Target (VM 2) without passwords, configure a dedicated deployment user:

```
On VM 2 (Target): Create user and grant passwordless sudo privileges:
sudo useradd -m -s /bin/bash deploy-user
echo 'deploy-user ALL=(ALL) NOPASSWD: ALL' | sudo tee /etc/sudoers.d/deploy-user
sudo mkdir -p /home/deploy-user/.ssh && sudo chmod 700 /home/deploy-user/.ssh

On VM 1 (Jenkins): Generate an SSH key pair as the jenkins service user:
sudo su - jenkins -s /bin/bash
ssh-keygen -t ed25519 -N '' -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub

Copy VM 1's public key and append it to /home/deploy-user/.ssh/authorized_keys on VM 2. Secure permissions:
sudo chmod 600 /home/deploy-user/.ssh/authorized_keys && sudo chown -R deploy-user:deploy-user
/home/deploy-user/.ssh
```

Verification: From VM 1 as jenkins user, run ssh deploy-user@<VM2-IP> — it should log in instantly without passwords.

3. Puppet Control Repo & r10k Setup

Create a GitHub control repository with directories: manifests/site.pp, hieradata/, and site-modules/.

```
Configure r10k on VM 2 at /etc/r10k.yaml:
---
:cachedir: '/opt/puppetlabs/server/data/r10k/cache'
:sources:
:my-control-repo:
:remote: 'git@github.com:<your-username>/puppet-control-repo.git'
:prefix: false
```

4. Creating the 3 Jenkins Jobs & Dependencies

Job 1: Puppet-r10k-Sync (Webserver & Infrastructure)

• *Type:* Freestyle project.

• *Build Step (Execute shell):* Pulls latest code via r10k and applies site.pp for Apache2:
ssh -o StrictHostKeyChecking=no deploy-user@<VM2-IP> 'sudo r10k deploy environment production -p --verbose && sudo puppet apply /etc/puppetlabs/code/environments/production/manifests/site.pp'

Job 2: Tomcat-Appserver-Deploy

• *Type:* Freestyle project.

• *Build Step (Execute shell):* Applies Puppet profile for Tomcat service configuration:
ssh -o StrictHostKeyChecking=no deploy-user@<VM2-IP> 'sudo puppet apply -e "include profile::tomcat"'

Job 3: HelloWorld-Maven-Build
• *Type:* Maven project (or Freestyle with Maven build step).
• *Build Step:* Root POM: pom.xml, Goals and options: clean package.
• *Post-build Step:* Copy generated target/helloworld.jar to Tomcat webapps or deployment directory on VM 2 via SCP.

Job Dependency Chain:
1. Open **Job 2 (Tomcat-Appserver-Deploy)** configuration -> Build Triggers -> Check *Build after other projects are built* -> Projects to watch: Puppet-r10k-Sync.
2. Open **Job 3 (HelloWorld-Maven-Build)** -> Check *Build after other projects are built* -> Projects to watch: Tomcat-Appserver-Deploy.

5. GitHub Webhook Integration

1. In Jenkins, install the **GitHub Plugin**.
2. Go to Job 1 (Puppet-r10k-Sync) configuration -> Build Triggers -> Check *GitHub hook trigger for GITScm polling*.
3. On GitHub repository -> Settings -> Webhooks -> Add webhook -> Payload URL: http://<VM1-Jenkins-IP>:8080/github-webhook/ -> Content type: application/json -> Events: Just the push event -> Add webhook.
Verification: Push a commit to GitHub; Jenkins will automatically trigger Job 1, which cascades sequentially to Job 2 and Job 3.